

OpenLeave — Architecture

OpenLeave (prototype)

August 2026

Contents

OpenLeave — Architecture	1
1. The big picture	1
2. The core, layer by layer	2
2.1 Facts — the typed input boundary	2
2.2 Parameters — effective-dated statutory values	2
2.3 Engine core — the shared vocabulary	2
2.4 Domain logic — regimes and topics	3
2.5 Cross-cutting honesty layers	3
3. Data flow (one determination)	3
4. The honesty invariants (why the core is shaped this way)	4
5. The amendment watcher (keeping the encoding current)	4
6. Two domains, one substrate	4
7. The verification manifest (the trust boundary)	5
8. The surfaces (edges as thin adapters)	5
9. Where to extend	5
10. Design in one breath	6

OpenLeave — Architecture

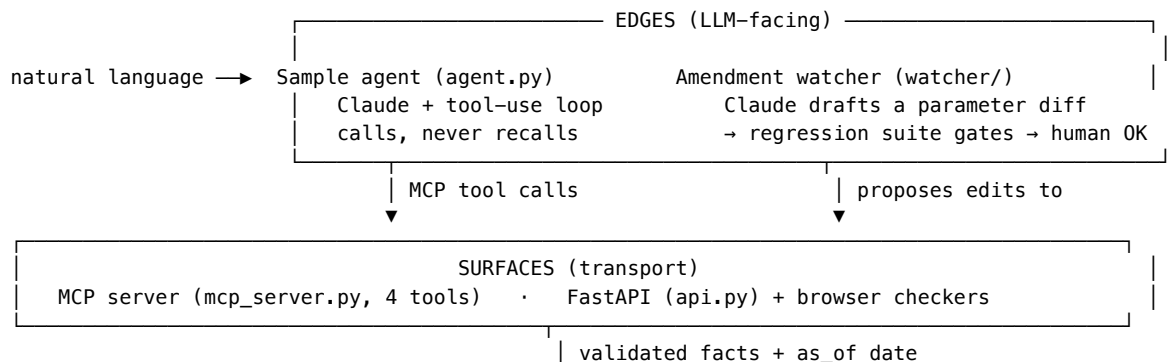
How the system is built, why it's built that way, and where to extend it.

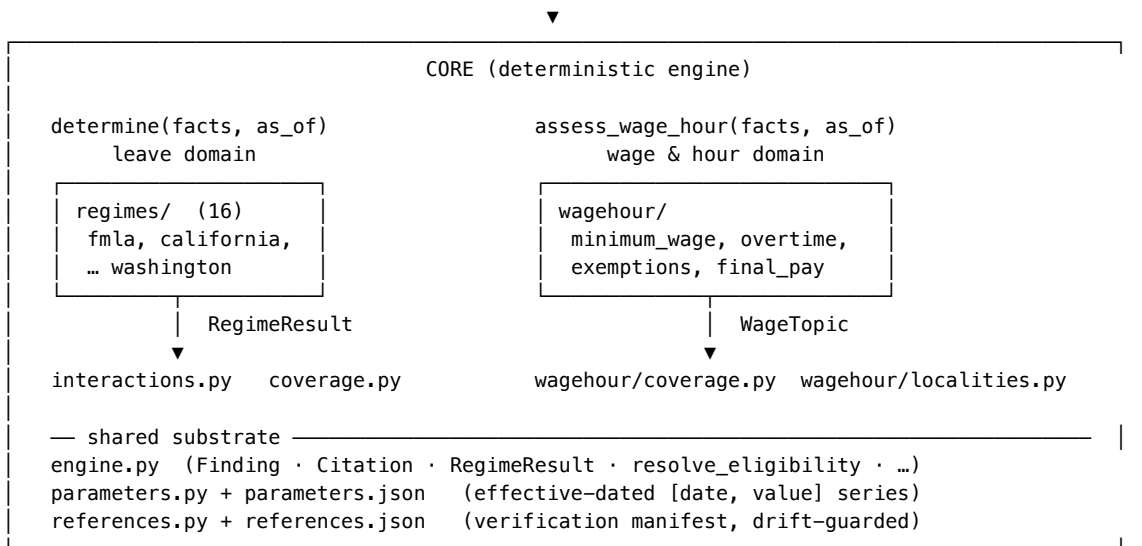
OpenLeave is an executable, citation-backed encoding of U.S. employment law (employee leave plus wage & hour). Its one organizing idea is the survey's thesis:

The LLM belongs at the edges; a verified rules engine belongs at the core.

Everything below follows from taking that seriously. Language models draft amendments and hold the conversation; they never decide the law. The core is deterministic, every conclusion carries its citation, genuine discretion is surfaced rather than guessed, and the engine reports the limits of its own coverage instead of bluffing.

1. The big picture





Three tiers: **edges** (anything that involves an LLM), **surfaces** (transport — MCP, HTTP, browser), and the **core** (a pure, deterministic Python library). The core has no knowledge of the surfaces or the LLM; it is a library you can call directly, and everything above it is a thin adapter.

2. The core, layer by layer

2.1 Facts — the typed input boundary

Inputs are Pydantic models, so validation is the API contract. Two fact shapes, because the two domains reason about different things:

- **Leave** (facts.py): Facts { employee: Employee, employer: Employer, event: LeaveEvent }. Leave is about a **discrete event** — a LeaveEvent with a start date and a LeaveReason.
- **Wage & hour** (wagehour/facts.py): WageFacts (pay rate, hours, salary, tipped status, optional Separation). Wage & hour is about **ongoing employment**, so it has no single “event.”

That difference is why wage & hour is a *sibling* entry point (assess_wage_hour) rather than more regimes bolted onto determine — see §6.

2.2 Parameters — effective-dated statutory values

parameters.json maps each key to a list of [effective_date, value] pairs; parameters.get(key, as_of) returns the value in force on a date:

```
"minwage.CA": [{"2025-01-01", 16.5}, {"2026-01-01", 16.9}]
```

This single mechanism gives the engine “**law as of any date.**” A determination dated 2025 sees \$16.50; the same facts dated 2026 see \$16.90. Reconstructing a past determination for an audit is just passing an earlier as_of. State-average-weekly-wage bumps, benefit caps, program launch dates, minimum-wage steps, and the vacated-then-restored federal exempt threshold are all just data points on a timeline.

The store also honors OPENLEAVE_PARAM_OVERRIDES — a path to a JSON file merged over the base data at import. That is the seam the amendment watcher uses to run the regression suite against a *proposed* change without touching the canonical data (§5).

2.3 Engine core — the shared vocabulary

engine.py defines the types both domains speak, so a leave finding and a wage finding are the same kind of object:

- **Citation** { ref, url } — a statute/regulation reference. Nothing is asserted without one.
- **Finding** { key, description, met: bool | None, citation, detail } — one atomic conclusion. met=None is load-bearing: it means *a human must decide this* (see §4).

- **Entitlement, RegimeResult** — a leave regime’s structured output (eligibility + entitlement + findings + human-judgment items + notes).
- **resolve_eligibility(findings)** — combines findings: False if any hard condition fails, None if the only open items need human judgment, else True. Discretion propagates; it is never silently resolved to a boolean.
- **encoded_range_note / not_yet_in_force_note** — the two ways “no determination” differs from “no entitlement” (§4).

wagehour/result.py adds **WageTopic** — the wage analogue of RegimeResult (a list of Findings + a data bag + human-judgment + notes), reusing the same Finding/Citation types.

2.4 Domain logic — regimes and topics

Leave. Each regime is a module in regimes/ exposing evaluate(facts, as_of) -> RegimeResult. determine() calls all of them, then layers on interactions and coverage. Anatomy of a regime (washington.py is representative):

```
evaluate(facts, as_of):
    applies?          work_state matches, else return inert result
    date gating       as_of < ENCODED_FROM -> applies=False + encoded_range_note
    reason filter      is this leave reason covered?
    build findings     each a cited Finding; open-textured ones get met=None + human_judgment
    resolve_eligibility(findings) -> True | False | None
    build Entitlement (weeks, job_protected, weekly_benefit, notes)
```

Wage & hour. Each topic is a module in wagehour/ exposing assess(facts, as_of) -> WageTopic | None (returning None when it doesn’t apply — e.g. no separation means no final-pay topic). assess_wage_hour() runs minimum wage, then optionally exemption, overtime, and final pay.

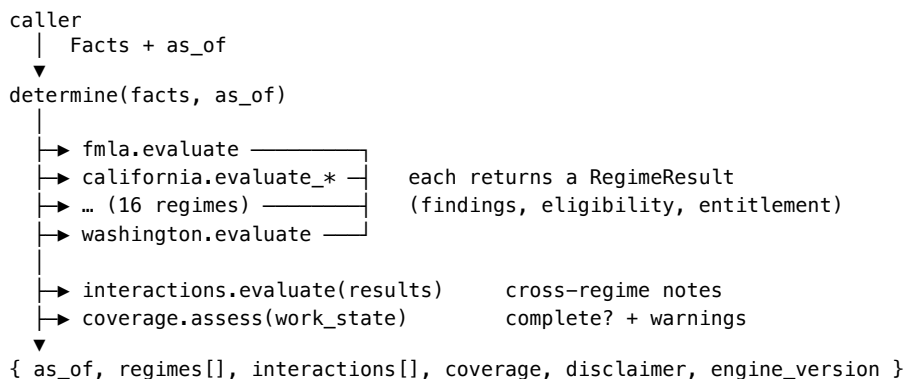
The two dispatchers are deliberately parallel:

```
determine(facts, as_of) -> {
    as_of, regimes[], interactions[],
    coverage, disclaimer, engine_version
}
assess_wage_hour(facts, as_of) -> {
    as_of, jurisdiction, topics[],
    coverage, disclaimer, engine_version
}
```

2.5 Cross-cutting honesty layers

- **Coverage** (coverage.py for leave, wagehour/coverage.py for wage) answers “is this the whole picture?” A jurisdiction with a program the engine doesn’t encode comes back complete: false with a loud warning. For wage, localities.py adds the city/county dimension: minimum wage is max(federal, state, local), an *encoded* locality is a complete answer, an *unencoded* one still warns (the engine can’t rule out an ordinance it doesn’t know).
- **Interactions** (interactions.py, leave only) computes cross-regime rules that no single regime can see: FMLA/state concurrency, pay-only + protection pairing, the DOL no-forced-stacking note.
- **Verification manifest** (references.py, §7).

3. Data flow (one determination)



assess_wage_hour is the same shape: fan out to topic assessors, attach locality-aware coverage, return a dict. Nothing mutates shared state; each assessor is a pure function of (facts, as_of) plus the read-only parameter store.

4. The honesty invariants (why the core is shaped this way)

These are the load-bearing design decisions. Each exists to prevent a specific failure mode of a “rules oracle.”

Invariant	Mechanism	Failure it prevents
Every conclusion cites its source Discretion is flagged, never compiled	<code>Finding.citation</code> is required <code>met=None + human_judgment;</code> <code>resolve_eligibility</code> propagates <code>None</code>	Unverifiable / hallucinated law Auto-answering open-textured questions (“serious health condition”, the exemption duties test) — the survey’s “laundered interpretation”
Coverage is reported, never assumed	<code>coverage.complete: false + warnings</code>	Silent under-coverage — a partial answer read as complete (the most dangerous failure)
Outside the encoded range ≠ no entitlement Enacted ≠ in force	<code>encoded_range_note</code> <code>not_yet_in_force_note</code> (e.g. MD FAMI, payable 2028)	Reading “we lack the historical rate” as “no benefit” Fabricating a benefit that can’t be claimed yet, or omitting the program entirely
Law as of any date	effective-dated parameters	Answering with today’s numbers for a past event

The two “N/A” notes are worth dwelling on because they’re mirror images: `encoded_range_note` means *the program was paying then, we just don’t encode that period’s rates*; `not_yet_in_force_note` means *the program is enacted but pays nothing yet*. Both are distinct from a denial, and the engine says which one applies.

5. The amendment watcher (keeping the encoding current)

Encoding the law once is easy; keeping it current is the actual product. The watcher is a human-in-the-loop pipeline that lets an LLM *propose* but never *decide*:

```
amendment doc → analyze (Claude, structured output) → proposal (parameter/logic diff + provenance)
                                     |
parameter change? → validate: run the FULL regression suite with the diff
                                     as an OPENLEAVE_PARAM_OVERRIDES overlay
structural change? → flagged requires_human_encoding (never auto-applied)
                                     |
                                     review (human approves, recorded) → apply (only if valid + approved)
```

Guarantees, enforced in code: forward-dated parameter updates pass; a diff that rewrites an in-force historical value breaks pinned determinations and is rejected; hallucinated parameter keys are rejected; nothing is applied without passing tests *and* an explicit, recorded human sign-off. The regression suite is the gate — the same property the effective-dated store makes cheap.

6. Two domains, one substrate

Wage & hour was added as a **sibling capability**, not a rewrite. It reuses the core wholesale — `engine.py`’s `Finding/Citation`, the effective-dated parameters store, the coverage-reporting reflex, and the verification manifest — and adds only what is genuinely different:

- a new fact shape (`WageFacts`, ongoing-state rather than event),
- a new entry point (`assess_wage_hour`, sibling to `determine`),
- a locality dimension (`wagehour/localities.py`),
- topic assessors instead of regimes.

This is the template for a third domain (e.g. meal/rest breaks, WARN-Act notice): keep the shared substrate, add a fact shape + entry point + assessors + manifest entries.

The coupling between exemptions and overtime is the one intentional intra-domain dependency: overtime is owed only to non-exempt workers, so `overtime.assess` consults `exemptions.salary_test` to state its result as conditional when a worker is *possibly* exempt (salary test met, duties test unresolved). Even here the duties test is never decided — it is passed through as human judgment.

7. The verification manifest (the trust boundary)

Every statutory value is web-researched and **unverified by counsel** — the one thing that must change before real use. `references.json` is the worksheet: it maps every parameter key to a plain-English meaning, the governing statute, and the agency URL to check it against, grouped by jurisdiction with sign-off fields (`verified` / `verified_by` / `verified_on`) and structural “claims” (logic that isn’t a single number).

The manifest is **drift-guarded**: a test asserts it documents *exactly* the set of encoded parameters. So a rate cannot be added without a citation (build fails), a removed parameter can’t leave stale docs behind, and a jurisdiction can’t be marked verified without a named reviewer and date. The MCP parameter-lookup tool surfaces each value’s citation and verification status, so a consumer always knows a figure is still pending review.

8. The surfaces (edges as thin adapters)

All three surfaces call the same two core functions and add nothing to the reasoning:

- **MCP server** (`mcp_server.py`) — four read-only tools, Pydantic-validated inputs, markdown or JSON output. Tool descriptions instruct the model to *call rather than recall* and document the three outcomes it must distinguish (a determined answer, `met/eligible: null` needing judgment, `coverage.complete: false` meaning partial).
- **REST API + browser checkers** (`api.py`, `checker.html`, `wage_checker.html`) — `POST /determinations`, `POST /wage-hour/determinations`, and two browser UIs.
- **Sample agent** (`agent.py`) — a Claude tool-use loop over the MCP server. The demonstration that the loop closes: the model converses, the engine decides, and each tool call is printed so the sourcing is visible. The agent is the *only* place a model is asked to answer a user, and even there it is forbidden from answering substantive questions except through the tools.

9. Where to extend

To add...	Touch...	The drift-guard / tests will force...
A leave regime	new <code>regimes/<state>.py</code> with <code>evaluate</code> ; register in <code>regimes/__init__.py</code> , <code>determine()</code> , <code>coverage</code> , <code>interactions</code>	a test file; manifest entries for its parameters
A wage topic	new <code>wagehour/<topic>.py</code> with <code>assess</code> ; wire into <code>assess_wage_hour</code>	tests; manifest entries
A jurisdiction / locality	<code>parameters.json</code> ; <code>coverage/localities</code>	manifest entries (or references check fails)
A single parameter	<code>parameters.json</code> and <code>references.json</code>	nothing else — but omitting the manifest entry fails CI

The recurring pattern: **the manifest and the tests make it impossible to add law without documenting its source**. That is the architecture’s way of enforcing its own central promise.

10. Design in one breath

A deterministic, effective-dated, citation-backed rules core; two parallel domains over one shared substrate; honesty invariants that surface discretion and coverage limits instead of hiding them; a human-gated LLM pipeline to stay current; and thin LLM/HTTP/browser edges that transport questions in and cited answers out — never deciding the law themselves.